

Compiling C with gcc

From .c file to running program — on Windows, macOS and Linux

CPE101 Teaching Assistant

CPE101 Computer Programming King Mongkut's University of Technology Thonburi (KMUTT)

Tutorial session

| Today

- What does a compiler do?
- Compiling a single file
- What happens inside gcc
- Programs with several files
- Teaser: make

| The computer cannot read C

You write **source code** (text). The CPU only runs **machine code** (bytes). A **compiler** translates one into the other.



- You compile **once**, then run the executable as many times as you like.
- Change the code $\Rightarrow$ you must **save and compile again**. The old .exe does not update itself.

TH คอมไพเลอร์ทำหน้าที่ “แปล” โค้ดภาษา C ที่คนอ่านออก ให้เป็นภาษาเครื่องที่ CPU รันได้ แก่โค้ดแล้วต้องเซฟและคอมไพล์ใหม่ทุกครั้ง

| Where do I type the commands?

The gcc commands are the **same on every OS**. Only the terminal differs.

Windows

Use the **MSYS2 UCRT64** terminal, or PowerShell / the VS Code terminal if C:\msys64\ucrt64\bin is on your PATH.

macOS

Use **Terminal**. Here gcc is really Apple **clang** under another name. That's fine: every command in this deck still works.

Linux

Use any terminal. gcc is the real GNU compiler, usually already installed (package build-essential).

Check that it works:

```
$ gcc --version
gcc.exe (Rev5, Built by MSYS2 project) 16.1.0
```

If you see “command not found” / “not recognized”, gcc isn’t on your PATH. On macOS the first line says Apple clang version ... instead.

| Just enough terminal to survive

The terminal always works **inside one folder**, the current directory. Your `.c` file has to be in it.

What	PowerShell (Windows)	macOS / Linux / MSYS2
Where am I?	<code>pwd</code>	<code>pwd</code>
List files	<code>ls (or dir)</code>	<code>ls</code>
Go into a folder	<code>cd lab01</code>	<code>cd lab01</code>
Go up one folder	<code>cd ..</code>	<code>cd ..</code>
Run a program here	<code>.\hello.exe</code>	<code>./hello</code>

Tip: avoid spaces and Thai characters in the folder names you keep code in, e.g. `C:\cpe101\lab01`.

TH ก่อนคอมไพล์ ต้อง `cd` เข้าไปในโฟลเดอร์ที่มีไฟล์ `.c` ก่อนเสมอ

| Your first compile

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello, CPE101!\n");
6     return 0;
7 }
```

hello.c

Simplest possible command:

```
$ gcc hello.c
```

| Your first compile

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello, CPE101!\n");
6     return 0;
7 }
```

hello.c

Simplest possible command:

```
$ gcc hello.c
```

No message means **success**. gcc only talks when something is wrong.

It made a file with a **default name**:

- Windows: a.exe
- macOS / Linux: a.out

Your first compile

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello, CPE101!\n");
6     return 0;
7 }
```

hello.c

Simplest possible command:

```
$ gcc hello.c
```

No message means **success**. gcc only talks when something is wrong.

It made a file with a **default name**:

- Windows: a.exe
- macOS / Linux: a.out

Run it:

```
$ ./a.exe      # Windows (PowerShell: .\a.exe)
$ ./a.out      # macOS / Linux
Hello, CPE101!
```

| Name your program with -o

`-o name` means “write the **output** to this file”.

```
$ gcc hello.c -o hello
```

	File created	How to run
Windows	hello.exe (.exe is added for you)	.\hello.exe (PowerShell), ./hello (MSYS2)
macOS / Linux	hello	./hello

TH ทำไมต้องมี ./ ? เพราะ macOS/Linux ไม่ค้นหาโปรแกรมในโฟลเดอร์ปัจจุบันให้อัตโนมัติ ต้องบอกตรง ๆ ว่า “โปรแกรมที่อยู่ตรงนี้”

| Name your program with -o

`-o name` means “write the **output** to this file”.

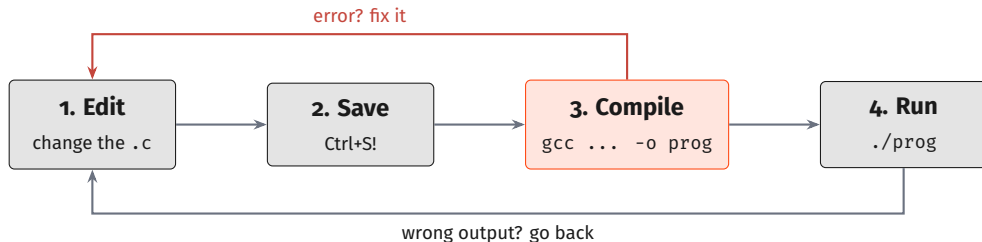
```
$ gcc hello.c -o hello
```

	File created	How to run
Windows	hello.exe (.exe is added for you)	.\hello.exe (PowerShell), ./hello (MSYS2)
macOS / Linux	hello	./hello

Common mistake

`gcc hello.c -o hello.c` **overwrites your source code** with the program! Never put the `.c` name after `-o`.

The edit-compile-run loop



“I fixed it but nothing changed!” The usual reasons:

- The file wasn't **saved** before compiling.
- You **didn't recompile**, so you ran the old .exe.
- Compiling **failed**, so the old .exe is still there. Read the messages!

| Turn on warnings: -Wall -Wextra

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int score;
6      float avg = 7.5;
7      printf("avg = %d\n", avg);
8      if (score = 10)
9          printf("ten!\n");
10     return 0;
11 }
```

Plain gcc warn.c compiles this
silently. But it's buggy.

Turn on warnings: -Wall -Wextra

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int score;
6     float avg = 7.5;
7     printf("avg = %d\n", avg);
8     if (score = 10)
9         printf("ten!\n");
10    return 0;
11 }
```

Plain gcc warn.c compiles this **silently**. But it's buggy.

```
$ gcc -Wall -Wextra warn.c -o warn
warn.c: In function 'main':
warn.c:7:20: warning: format '%d' expects argument of
      type 'int', but argument 2 has type 'double' [-Wformat=]
      7 |     printf("avg = %d\n", avg);
        |                ^~
        |                |
        |                int  double
        |                %f
warn.c:8:9: warning: suggest parentheses around
      assignment used as truth value [-Wparentheses]
      8 |     if (score = 10)
        |         ^~~~~~
```

Two real bugs, found for free. **Always** compile with -Wall -Wextra: a warning still produces a program, but it's usually a bug waiting to happen.

| Other flags worth knowing

Flag	Meaning
-o name	name of the output file
-Wall -Wextra	turn on (almost) all the useful warnings
-std=c11	use the C11 standard. gcc 15+ defaults to C23, so pick one and use the same as the lab
-lm	link the math library, needed for sqrt, pow, sin, ... on Linux. Harmless on Windows/macOS. Put it at the end
-g	keep debugging info for gdb (see the debugging session)

Everyday command, copy this

```
gcc -Wall -Wextra -std=c11 prog.c -o prog -lm
```

Example: a program that uses `<math.h>`

```
1  #include <stdio.h>
2  #include <math.h>
3
4  #define PI 3.14159265358979
5
6  int main(void)
7  {
8      double r = 2.0;
9      double area = PI * pow(r, 2);
10     printf("area = %.2f\n", area);
11     printf("sqrt(2) = %.4f\n", sqrt(2.0));
12     return 0;
13 }
```

```
$ gcc -Wall -Wextra -std=c11 circle.c
   -o circle -lm
$ ./circle
area = 12.57
sqrt(2) = 1.4142
```

Example: a program that uses `<math.h>`

```
1 #include <stdio.h>
2 #include <math.h>
3
4 #define PI 3.14159265358979
5
6 int main(void)
7 {
8     double r = 2.0;
9     double area = PI * pow(r, 2);
10    printf("area = %.2f\n", area);
11    printf("sqrt(2) = %.4f\n", sqrt(2.0));
12    return 0;
13 }
```

```
$ gcc -Wall -Wextra -std=c11 circle.c
  -o circle -lm
$ ./circle
area = 12.57
sqrt(2) = 1.4142
```

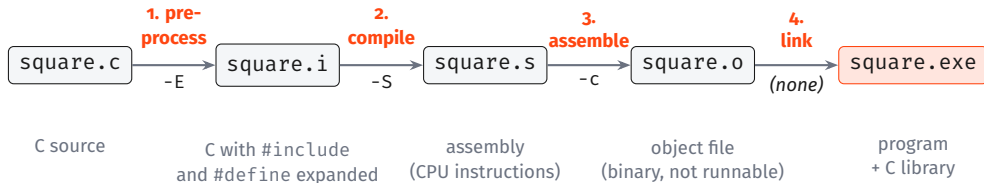
Without `-lm`, on Linux you may get:

```
undefined reference to `pow'
```

That's a **linker** error: the function exists, but its library wasn't linked in. More on the linker soon.

| One command, four stages

gcc is really a **driver** that runs four tools in a row:



The flag under each arrow means “**stop after this stage**”: `-E` prints the preprocessed C, `-S` writes `square.s`, `-c` writes `square.o`. With no flag, gcc goes all the way.

TH ปกติเราไม่เห็นขั้นตอนเหล่านี้ เพราะ gcc ทำให้ครบทั้ง 4 ขั้นตอนในคำสั่งเดียว

Peek at each stage

```
1 #include <stdio.h>
2 #define SIZE 3
3
4 int main(void)
5 {
6     int n = SIZE * SIZE;
7     printf("%d\n", n);
8     return 0;
9 }
```

square.c

1. Preprocess: gcc -E square.c (last lines)

```
# 4 "square.c"
int main(void)
{
    int n = 3 * 3;
    printf("%d\n", n);
    return 0;
}
```

SIZE became 3. `#include <stdio.h>` was pasted in as *hundreds* of lines above this.

Peek at each stage

```
1 #include <stdio.h>
2 #define SIZE 3
3
4 int main(void)
5 {
6     int n = SIZE * SIZE;
7     printf("%d\n", n);
8     return 0;
9 }
```

square.c

2. Compile: `gcc -S square.c` → `square.s`

```
main:
    pushq    %rbp
    movq     %rsp, %rbp
    subq     $48, %rsp
    call     __main
    movl     $9, -4(%rbp)
    ...
```

Assembly for an x86-64 CPU. Look: \$9. The compiler already worked out $3 * 3$.

Peek at each stage

```
1 #include <stdio.h>
2 #define SIZE 3
3
4 int main(void)
5 {
6     int n = SIZE * SIZE;
7     printf("%d\n", n);
8     return 0;
9 }
```

square.c

3. Assemble: `gcc -c square.c` → `square.o`

A binary **object file**: machine code, but `printf` is still just a *name* that nobody has filled in yet. You can't run it.

4. Link: `gcc square.o -o square`

The **linker** joins your `.o` files with the C library (where `printf` actually lives) into one runnable program.

Want to keep every intermediate file?

`gcc -save-temps square.c`

Why should I care? Errors come from different stages

Stage	Typical message	Usual cause
Preprocessor	fatal error: stdoi.h: No such file ...	typo in #include
Compiler	error: expected ',' or ';' before ...	syntax error in your code
Compiler	warning: unused variable 'x'	suspicious code (still compiles)
Linker	undefined reference to 'square'	function never defined, or a .c file / -lm missing
Linker	multiple definition of 'square'	same function defined in two files

Key idea

file.c:LINE:COL ⇒ **compiler**: go to that line. ld / collect2 ⇒ **linker**: check *which files* you gave gcc.

| Why split a program into files?

- **Organise:** one file per topic (math, input, display...).
- **Reuse:** use the same functions in many programs.
- **Teamwork:** people edit different files.
- **Faster builds:** recompile only what changed.

TH โปรแกรมใหญ่ ๆ ไม่เขียนรวมไฟล์เดียว แยกเป็นหลายไฟล์ตามหน้าที่

File	Contains
mathutil.h	declarations (prototypes): <i>what exists</i>
mathutil.c	definitions: <i>how it works</i>
main.c	main(), which uses them

A .h file is like a **menu**: it lists the dishes. The .c file is the **kitchen** that cooks them.

I The three files

mathutil.h

```
#ifndef MATHUTIL_H
#define MATHUTIL_H

int square(int x);
int cube(int x);

extern int call_count;

#endif
```

mathutil.c

```
#include "mathutil.h"

int call_count = 0;

/* private to this file */
static int mul(int a, int b)
{
    return a * b;
}

int square(int x)
{
    call_count++;
    return mul(x, x);
}

int cube(int x)
{
    call_count++;
    return mul(x, mul(x, x));
}
```

main.c

```
#include <stdio.h>
#include "mathutil.h"

int main(void)
{
    printf("square(5) = %d\n",
           square(5));
    printf("cube(3)   = %d\n",
           cube(3));
    printf("calls     = %d\n",
           call_count);
    return 0;
}
```

<stdio.h>: system header (angle brackets)

"mathutil.h": your own header (quotes, looked up in this folder first)

| Way 1: compile everything in one command

List **all the .c files**. Never list the .h files: they get #included.

```
$ gcc -Wall -Wextra -std=c11 main.c mathutil.c -o app
$ ./app
square(5) = 25
cube(3)    = 27
calls      = 2
```

| Way 1: compile everything in one command

List **all the .c files**. Never list the .h files: they get #included.

```
$ gcc -Wall -Wextra -std=c11 main.c mathutil.c -o app
$ ./app
square(5) = 25
cube(3)    = 27
calls      = 2
```

Forget a file? The compiler is happy, but the **linker** isn't:

```
$ gcc main.c -o app
ld.exe: ...main.c:(.text+0x13): undefined reference to `square'
collect2.exe: error: ld returned 1 exit status
```

Way 1: compile everything in one command

List **all the .c files**. Never list the .h files: they get #included.

```
$ gcc -Wall -Wextra -std=c11 main.c mathutil.c -o app
$ ./app
square(5) = 25
cube(3)    = 27
calls      = 2
```

Forget a file? The compiler is happy, but the **linker** isn't:

```
$ gcc main.c -o app
ld.exe: ...main.c:(.text+0x13): undefined reference to `square'
collect2.exe: error: ld returned 1 exit status
```

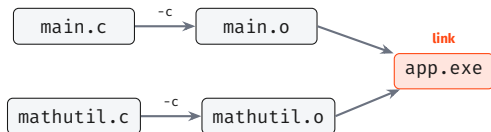
Common mistake

#include "mathutil.c". Never include a .c file: pass it to gcc instead.

Way 2: compile separately, then link

```
$ gcc -Wall -Wextra -std=c11 -c main.c
$ gcc -Wall -Wextra -std=c11 -c mathutil.c
$ gcc main.o mathutil.o -o app
$ ./app
square(5) = 25
...
```

-c = compile only, **don't link**. Each .c becomes a .o.



Way 2: compile separately, then link

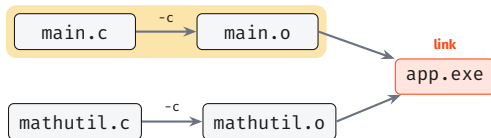
```
$ gcc -Wall -Wextra -std=c11 -c main.c
$ gcc -Wall -Wextra -std=c11 -c mathutil.c
$ gcc main.o mathutil.o -o app
$ ./app
square(5) = 25
...
```

-c = compile only, **don't link**. Each .c becomes a .o.

Changed only main.c? Redo just two steps:

```
$ gcc -Wall -Wextra -std=c11 -c main.c
$ gcc main.o mathutil.o -o app
```

In a project with 100 files, this saves a lot of time.



Highlighted: the only part that is rebuilt. (mathutil.o is reused as-is.)

| Header guards: #ifndef ... #endif

A header can easily get included **twice** (directly, or through another header).
Without a guard, its contents are pasted twice:

```
#include "point.h"  
#include "point.h" /* oops */
```

```
$ gcc -std=c11 main.c  
point.h:1:8: error: redefinition of 'struct point'
```

TH header guard กัน “แปะซ้ำ” เวลา #include ไฟล์เดียวกันหลายครั้ง ใส่ทุกไฟล์ .h จนเป็นนิสัย

Header guards: #ifndef ... #endif

A header can easily get included **twice** (directly, or through another header). Without a guard, its contents are pasted twice:

```
#include "point.h"
#include "point.h" /* oops */
```

```
$ gcc -std=c11 main.c
point.h:1:8: error: redefinition of 'struct point'
```

Fix: wrap every header like this

```
1  #ifndef POINT_H      /* first time? */
2  #define POINT_H      /* remember it */
3
4  struct point { int x, y; };
5
6  #endif               /* 2nd time: skip
                        all */
```

The first include defines POINT_H. Any later include sees it's already defined and skips everything.

TH header guard กัน “แปะซ้ำ” เวลา #include ไฟล์เดียวกันหลายครั้ง ใส่ทุกไฟล์ .h จนเป็นนิสัย

Sharing between files: extern and static

Written as	In file	Meaning
<code>int call_count = 0;</code>	<code>mathutil.c</code>	the variable really lives here (definition, outside any function)
<code>extern int call_count;</code> <code>int square(int x);</code>	<code>mathutil.h</code> <code>mathutil.h</code>	"it exists somewhere else ": no memory is made here function prototype: functions are shareable by default
<code>static int mul(...)</code>	<code>mathutil.c</code>	private : other files can't see it

TH extern = ประกาศว่ามีอยู่ที่อื่น, static (นอกฟังก์ชัน) = ใช้ได้เฉพาะในไฟล์นี้เท่านั้น (เชื่อมกับข้อ 2 ในข้อสอบ mock)

Sharing between files: extern and static

Written as	In file	Meaning
<code>int call_count = 0;</code>	<code>mathutil.c</code>	the variable really lives here (definition, outside any function)
<code>extern int call_count;</code> <code>int square(int x);</code>	<code>mathutil.h</code> <code>mathutil.h</code>	"it exists somewhere else ": no memory is made here function prototype: functions are shareable by default
<code>static int mul(...)</code>	<code>mathutil.c</code>	private : other files can't see it

Calling the static function `mul` from another file fails at link time (undefined reference to `'mul'`). That's static doing its job.

TH `extern` = ประกาศว่ามีอยู่ที่อื่น, `static` (นอกฟังก์ชัน) = ใช้ได้เฉพาะในไฟล์นี้เท่านั้น (เชื่อมกับข้อ 2 ในข้อสอบ mock)

| Let make type the commands for you

A Makefile lists **target: what it needs**, then the command:

```
CC      = gcc
CFLAGS = -Wall -Wextra -std=c11

app: main.o mathutil.o
^^I$(CC) main.o mathutil.o -o app

main.o: main.c mathutil.h
^^I$(CC) $(CFLAGS) -c main.c

mathutil.o: mathutil.c mathutil.h
^^I$(CC) $(CFLAGS) -c mathutil.c

clean:
^^Irm -f *.o app app.exe
```

The indented lines must start with a TAB, not spaces.

```
$ make          # MSYS2: mingw32-make
gcc -Wall -Wextra -std=c11 -c main.c
gcc -Wall -Wextra -std=c11 -c mathutil.c
gcc main.o mathutil.o -o app
```

| Let make type the commands for you

A Makefile lists **target: what it needs**, then the command:

```
CC      = gcc
CFLAGS = -Wall -Wextra -std=c11

app: main.o mathutil.o
^^I$(CC) main.o mathutil.o -o app

main.o: main.c mathutil.h
^^I$(CC) $(CFLAGS) -c main.c

mathutil.o: mathutil.c mathutil.h
^^I$(CC) $(CFLAGS) -c mathutil.c

clean:
^^Irm -f *.o app app.exe
```

The indented lines must start with a TAB, not spaces.

```
$ make # MSYS2: mingw32-make
gcc -Wall -Wextra -std=c11 -c main.c
gcc -Wall -Wextra -std=c11 -c mathutil.c
gcc main.o mathutil.o -o app
```

Edit `main.c`, run make again:

```
$ make
gcc -Wall -Wextra -std=c11 -c main.c
gcc main.o mathutil.o -o app
```

make compares file times and **rebuilds only what changed**: exactly “Way 2”, done for you.

I Cheat sheet

I want to...	Command
compile one file ...that uses <math.h> run it	<code>gcc -Wall -Wextra -std=c11 prog.c -o prog</code> <code>add -lm at the end</code> <code>./prog (PowerShell: .\prog.exe)</code>
compile several files	<code>gcc main.c util.c -o app</code>
compile only (no link)	<code>gcc -c util.c → util.o</code>
link object files	<code>gcc main.o util.o -o app</code>
see preprocessed / assembly	<code>gcc -E prog.c / gcc -S prog.c</code>
prepare for the debugger	<code>add -g</code>

Rules of thumb

Always `-Wall -Wextra` • never `-o a .c name` • give gcc the `.c` files, never the `.h` • fix the **first** error first

| Try it yourself

1. Compile and run `hello.c` with `-o hello`. Then change the message **without** recompiling and run it again. What happens?
2. Compile `warn.c` with and without `-Wall -Wextra`. Fix both warnings.
3. Run `gcc -E square.c > square.i` and open `square.i`. How many lines did `#include <stdio.h>` add?
4. In `multi/`: add a function `int power(int b, int e)` to `mathutil.c` and `mathutil.h`, call it from `main.c`, and rebuild with “Way 2”.
5. Remove `mathutil.c` from the `gcc` command. Read the error. Which stage complained?

All example files are in `01-compiler/examples/`.